Neural Networks

Lecture 2 — Multilayer Networks

Last lecture: one neuron draws one line.

You tried. XOR beat you.

Let's fix that.

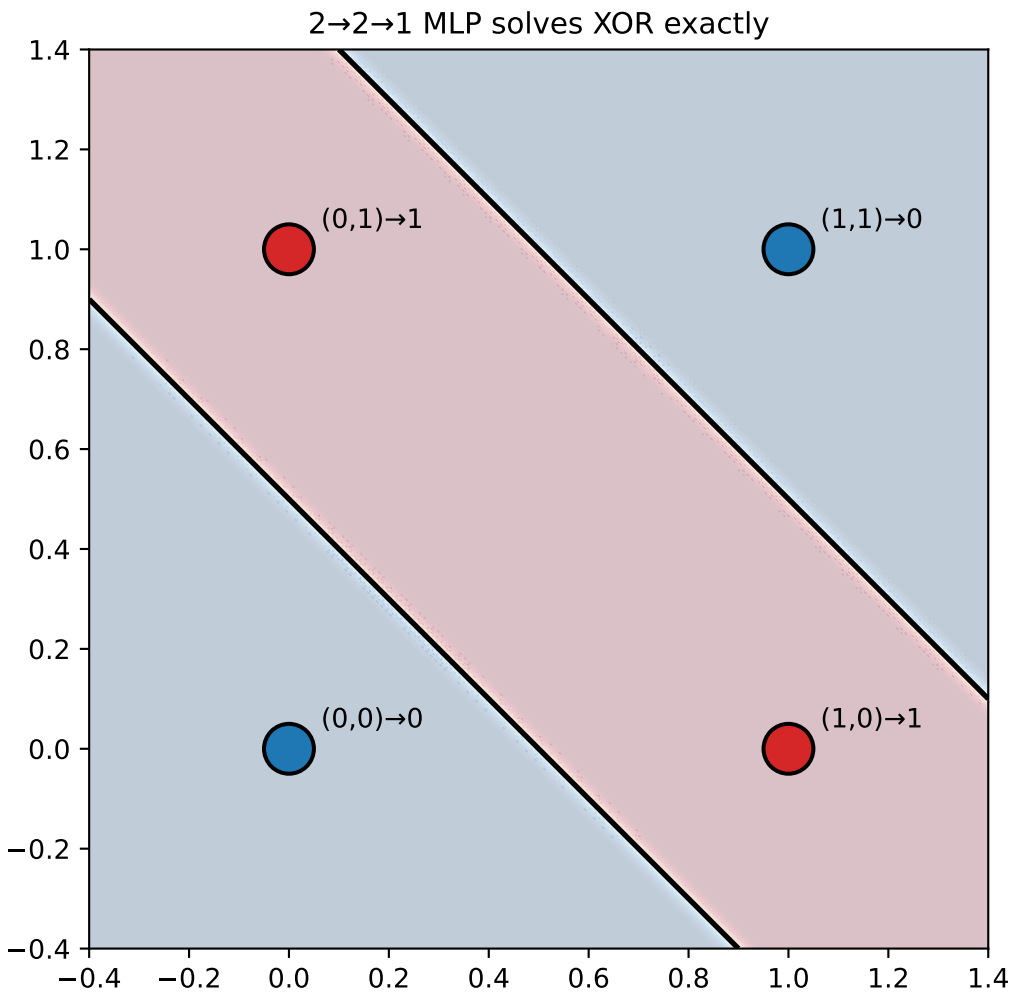
XOR — resolved

The four XOR points are not linearly separable.

But two lines can carve the plane correctly.

- Line 1: separates "at least one input on" from "both off"
- Line 2: separates "both on" from everything else

One hidden layer with two neurons draws two lines.



From one neuron to a network

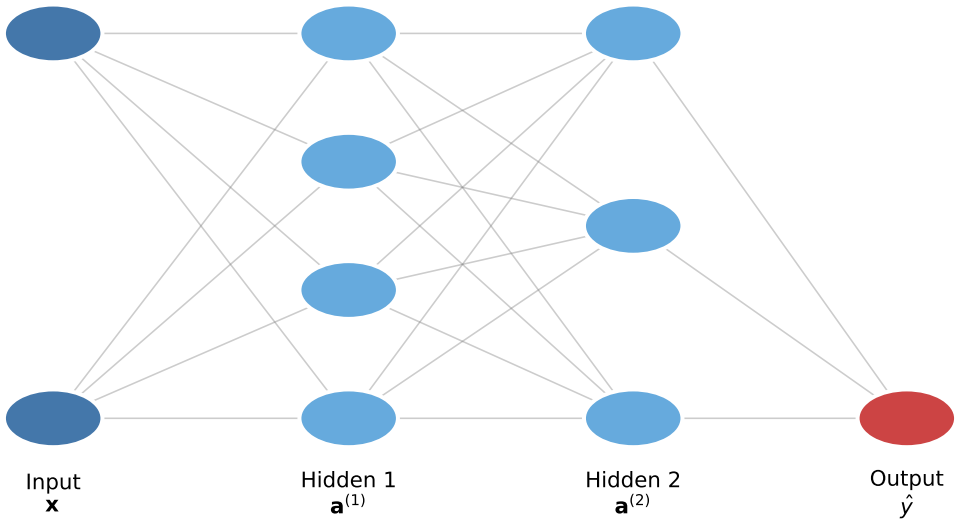
A multilayer perceptron (MLP) stacks layers of neurons.

$$\mathbf{a}^{(1)} = \sigma\big(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\big)$$
$$\mathbf{a}^{(2)} = \sigma\big(W^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}\big)$$
$$\hat{y} = \sigma\big(W^{(3)}\mathbf{a}^{(2)} + \mathbf{b}^{(3)}\big)$$

Each layer transforms the representation from the previous one.

Key: without σ, all layers collapse to a single linear map.

2 → 4 → 3 → 1 MLP



MLP architecture in PyTorch

```python
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, n_hidden):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, n_hidden),   # input → hidden
            nn.ReLU(),
            nn.Linear(n_hidden, 1),   # hidden → output
            nn.Sigmoid(),
        )

    def forward(self, x):
        return self.net(x)
```

`nn.Sequential` chains layers — each layer's output is the next layer's input. `nn.Linear(in, out)` holds a weight matrix $W \in \mathbb{R}^{\text{out}\times\text{in}}$ and bias $\mathbf{b} \in \mathbb{R}^{\text{out}}$.

Forward propagation — step by step

```
x = torch.tensor([[0., 1.]])   # one XOR input, shape (1, 2)

# Layer 1
z1 = x @ W1.T + b1             # (1,2) @ (2,4)   → (1,4)
a1 = torch.relu(z1)            # element-wise ReLU

# Layer 2
z2 = a1 @ W2.T + b2            # (1,4) @ (4,1)   → (1,1)
y_hat = torch.sigmoid(z2)      # probability
```

Shape accounting matters.

At every layer, check: does `(batch, in) @ (in, out)` give `(batch, out)`?

Getting this wrong is the most common source of errors when building networks.

Batch processing is free.

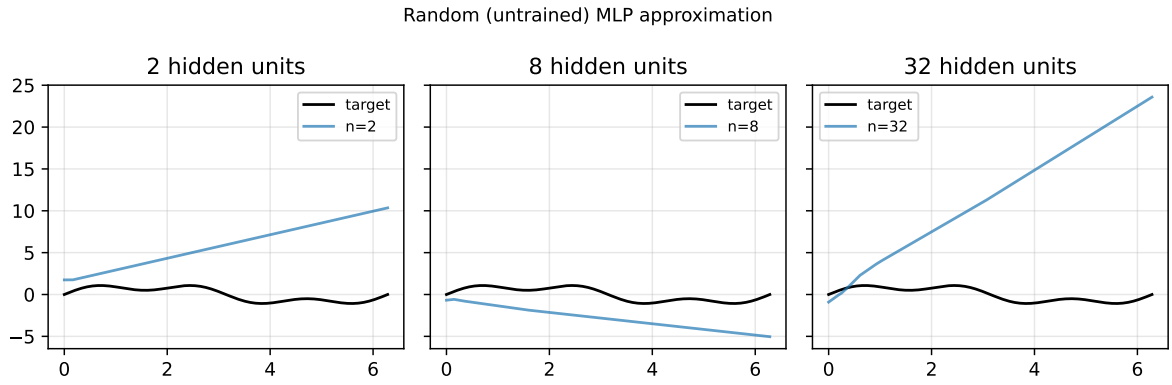
Replace the single input `(1, 2)` with a batch `(N, 2)`. The matrix multiply handles all N samples simultaneously — this is why GPUs matter.

What can an MLP represent?

> **Note**
>
> **Universal Approximation Theorem**
> A feedforward network with a single hidden layer containing a sufficient number of neurons with a non-linear activation can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary precision.

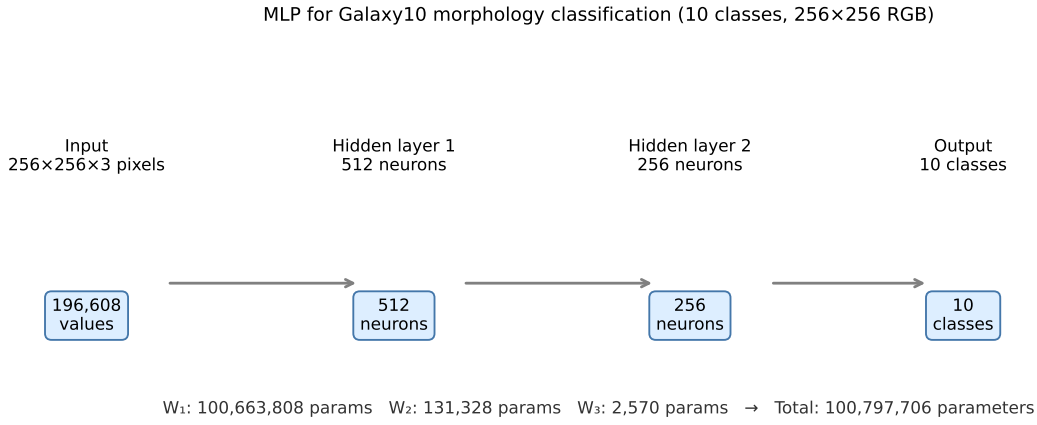
What it means: depth is not strictly necessary — width can compensate.

What it does not mean: "sufficient" neurons may be astronomically many. Depth is far more practical than width for complex functions.



Untrained — just to show the space of functions. Training finds the right weights.

Galaxy morphology — the parameter count problem



~**100 million parameters** to classify a single image type. Every pixel connected to every neuron — no notion of spatial structure. Neighbour pixels carry related information; the MLP ignores this entirely.

What MLPs cannot do well

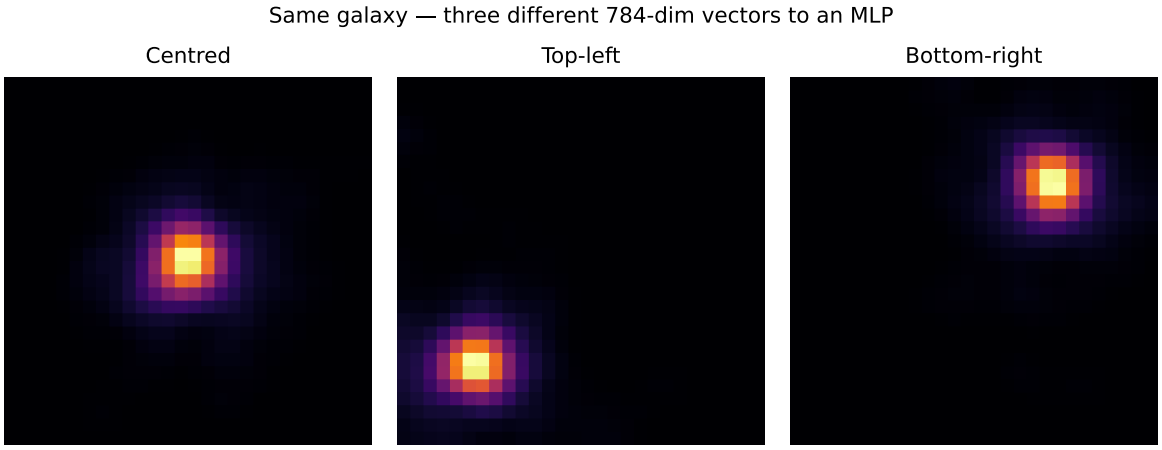
An MLP flattens every image into a vector before processing.

Lost: spatial structure — which pixels are neighbours

Lost: translation invariance — a spiral galaxy shifted by 10 pixels is a completely different input

Lost: local feature hierarchy — edges → textures → morphology

The right tool reuses the same filter at every position. That is a convolutional network. Lecture 4.



The bridge: learning the weights

We now know *what* the network computes (forward pass).

We do not yet know *how* it learns the weights.

The answer is the same gradient descent from the UL week — extended to arbitrarily deep compositions of functions.

> Tip
>
> **Open the notebook.** You will implement the full training loop in NumPy — forward pass, loss, gradients, update — and watch it converge on a photometric redshift estimator. Parts 2–4 use interactive sliders to explore learning rate, batch size, and overfitting.

References

Papers

- Cybenko (1989) — *Approximation by superpositions of a sigmoidal function.* Mathematics of Control, Signals and Systems 2, 303–314
- Hornik (1991) — *Approximation capabilities of multilayer feedforward networks.* Neural Networks 4(2), 251–257

Videos & blogs

- 3Blue1Brown — *Gradient descent, how neural networks learn* youtube.com/watch?v=IHZwWFHWaw
- Karpathy — *Hacker's guide to Neural Networks* karpathy.github.io/neuralnets/ — Chapter 2